

Project Scope: RPI-HTM Raspberry Pi Integration

1) Objective

Deploy the current C++ HTM SWAT pipeline on Raspberry Pi-class hardware and deliver a repeatable benchmark package by June 1, 2026.

Primary outcome:

- A reproducible build and run workflow for ARM64 (emulated now, physical Pi when available).
- Verified anomaly detection output on SWAT data.
- Resource and timing evidence comparable to literature baselines.

2) Competitor Analysis (from provided paper)

Source analyzed:

- A lightweight All-MLP time-frequency anomaly detection for IIoT time series (Neural Networks 187, 2025).

Key findings relevant to Raspberry Pi 4b

From Table 5 (Raspberry Pi 4b, ARM Cortex-A72, 2 GB RAM):

- LTFAD: 100-timestamps-time 73.3 s, CPU 31.8%, RAM 182 MB.
- DIFFI: 23.1 s, CPU 27%, RAM 102 MB (fastest among listed on Pi in this table).
- COUTA: 93.2 s, CPU 37.3%, RAM 192 MB.
- STEN: 96.4 s, CPU 64.3%, RAM 213 MB.
- SimAD: 204.2 s, CPU 42.7%, RAM 215 MB.
- TFMAE: 140.5 s, CPU 41.9%, RAM 203 MB.
- DCdetector: 425.1 s, CPU 72.2%, RAM 402 MB.
- ATF-UAD: 361.2 s, CPU 68.6%, RAM 343 MB.
- DTAAD: 328.6 s, CPU 67.7%, RAM 318 MB.
- PatchAD: Ot (out of memory).

SWaT detection context (Table 4)

- LTFAD on SWaT: ACC 0.9944, Precision 0.9557, Recall 1.0000, F-Score 0.9774.

Implications for your HTM-on-Pi plan

- Avoid deep-model resource profiles; they risk OOM and high CPU on 2 GB systems.
- Your near-term target should be stable runtime and predictable memory before trying to match best accuracy.
- A practical competitive baseline for Pi-4b is around:
 - RAM under 250 MB (good), under 350 MB (acceptable).
 - CPU under 45% average (good), under 65% (acceptable).
 - 100-timestamp processing under 120 s (acceptable), under 80 s (competitive).

Current HTM baseline snapshot (2 trusted runs)

Source runs:

- results/experiments_cpp_naive_20260115_115556_189
- results/experiments_cpp_naive_20260115_115321_175

Per-run values (from model_efficiency_metrics.json):

Run	total_runtime_ms	cpu.avg	ram.avg (MB)
---	---:	---:	---:
experiments_cpp_naive_20260115_115556_189	631832.000	12.716	3993.281
experiments_cpp_naive_20260115_115321_175	550740.000	12.704	4891.832

Arithmetic average across the 2 runs:

- Elapsed time average: 591286.000 ms (591.286 s, 9.855 min)
- CPU average (CCPU): 12.710%

3) Project Requirements (what is needed, and why)

A) Functional requirements

1. ARM64 build succeeds for htm.core and cppproject.

Why: Raspberry Pi 4b is ARM-based, and x86-only validation is not sufficient.

2. End-to-end SWAT run produces anomaly scores and metrics artifacts.

Why: Need functional parity with your current x86 pipeline.

3. Artifact outputs remain structured and reproducible.

Why: Enables comparison over tuning iterations and against competitor numbers.

B) Non-functional requirements

1. No OOM on 2 GB memory profile.

Why: Competitor shows OOM risk for heavier models on Pi.

2. Deterministic run configuration (fixed seed, fixed dataset slice).

Why: Benchmark fairness and regression tracking.

3. Stable profiling protocol (CPU, RAM, runtime, latency).

Why: You cannot improve what you cannot measure consistently.

C) Tooling requirements

1. Emulator stack for ARM64 Pi-like runs.

Why: Physical hardware is currently unavailable.

2. Cross-build or native ARM build workflow.

Why: Needed to iterate quickly now and transfer to real Pi later.

3. Automated benchmark scripts and result capture.

Why: Reduces manual error and saves time against deadline.

D) Data and evaluation requirements

1. SWAT dataset availability in emulator/target environment.

Why: Required to evaluate production-like workload.

2. Ground-truth labels wired for real metric reporting.

Why: Current synthetic labels should not be final for competitor comparison.

3. Consistent metric definitions (ACC, Precision, Recall, F1, plus resource metrics).

Why: Required for apple-to-apple comparisons.

4) Ordered Steps (tasks, requirements, outputs)

Step 1: Freeze benchmark contract

Requirements:

- Define exact dataset split, seed, sampling interval, and scoring protocol.

Why:

- Prevents moving targets during optimization.

Output:

- One benchmark contract document and run command set.

Step 2: Baseline current x86 results

Requirements:

- Run current executable in Release mode with existing monitor outputs.

Why:

- Gives a reference point before ARM migration.

Output:

- Baseline folder under results with performance and efficiency JSON/TXT.

Step 3: Prepare ARM64 build pipeline

Requirements:

- CMake toolchain config for ARM64 and dependency list for htm.core + yaml-cpp.

Why:

- Ensures code compiles for target architecture.

Output:

- Reproducible build instructions and successful ARM64 binary build.

Step 4: Emulator bring-up

Requirements:

- QEMU-based ARM64 VM profile configured to resemble Pi-4b class resources.

Why:

- Enables immediate pre-hardware validation.

Output:

- Bootable emulator with SSH access and repeatable launch script.

Step 5: Functional validation in emulator

Requirements:

- Run HTM pipeline end-to-end and confirm outputs are generated.

Why:

- Verifies behavior parity under ARM environment.

Output:

- Successful run artifacts in emulator environment.

Step 6: Integrate objective profiling pipeline

Requirements:

- Collect runtime, CPU, RAM, and latency at fixed interval.

Why:

- Needed to compare with competitor Table 5 style metrics.

Output:

- Versioned metrics files and summary table per run.

Step 7: Optimization pass 1 (low-risk)

Requirements:

- Compiler optimizations, build flags, and obvious data-path improvements.

Why:

- Quick performance wins before algorithmic changes.

Output:

- Reduced runtime and/or memory with changelog of what improved.

Step 8: Optimization pass 2 (algorithm/system)

Requirements:

- Profiling-guided hotspots (encoding, SDR ops, per-row loop, memory layout).

Why:

- Needed for stronger competitiveness under 2 GB constraints.

Output:

- Improved throughput and stable memory profile.

Step 9: Comparator-ready evaluation package

Requirements:

- Produce SWAT detection metrics and resource metrics in one consolidated report.

Why:

- Makes direct comparison to competitor claims possible.

Output:

- Comparison matrix HTM vs LTFAD reference figures.

Step 10: Real Pi readiness handoff

Requirements:

- Deployment checklist for first day hardware arrival.

Why:

- Avoids delays when Pi becomes available.

Output:

- Hardware day-1 script set: install, build, run, measure, report.

Step 11: Buffer and final sign-off

Requirements:

- Time reserved for regressions, reproducibility checks, and documentation polish.

Why:

- Deadline safety and quality assurance.

Output:

- Final release candidate and complete project report.
-

5) Timeline to June 1, 2026

Current date: March 28, 2026

Deadline: June 1, 2026

Total window: about 9 weeks + 2 days

Phase timeline

1. March 28 to April 3

Scope lock and baseline

- Deliverables: benchmark contract, x86 baseline artifacts.

2. April 4 to April 12

ARM64 build path and dependency hardening

- Deliverables: successful ARM64 build docs and binary.

3. April 13 to April 24

Emulator setup and first end-to-end ARM runs

- Deliverables: emulator launch scripts, first ARM run artifacts.

4. April 25 to May 8

Profiling and optimization pass 1

- Deliverables: measurable improvement in runtime and/or RAM.

5. May 9 to May 19

Optimization pass 2 + comparator report draft

- Deliverables: refined performance profile, first competitor comparison sheet.

6. May 20 to May 27

Acceptance runs and reproducibility checks

- Deliverables: repeated runs, variance summary, final metric tables.

7. May 28 to June 1

Buffer and final packaging

- Deliverables: final report, deployment guide, hardware day-1 checklist.

Weekly cadence recommendation

- Monday: planning and experiment setup.
 - Tuesday to Thursday: implementation and tuning.
 - Friday: benchmark runs and report update.
 - Weekend buffer: regression fixes or spillover.
-

6) Emulator Strategy (no physical Pi yet)

Recommended emulator choice

Use QEMU ARM64 as the primary emulator.

Reason:

- Best open-source option for Linux and Windows.
- Supports controlled CPU/RAM profiles close to Pi-class constraints.
- Integrates well with CI and scripting.

Important note:

- Emulation is useful for functional validation and relative tuning.
- Final timing and thermals must still be confirmed on real hardware.

Option A (recommended): Full-system ARM64 VM with QEMU

Best for:

- End-to-end deployment rehearsals and package verification.

Setup outline:

1. Install QEMU and tools
 - Linux: qemu-system-aarch64, qemu-utils.
 - Windows: QEMU for Windows (or run QEMU inside WSL2).
2. Create ARM64 VM image (Ubuntu Server ARM64 or Raspberry Pi OS 64-bit compatible flow).
3. Configure VM resources to mimic Pi-4b constraints:
 - 4 vCPU class, 2 GB RAM target profile.
4. Enable SSH port forwarding.
5. Run build + benchmark scripts inside emulator.

Output:

- Reusable emulator launch and test scripts.

Option B: Container-based ARM emulation for fast build checks

Best for:

- Rapid compile/test loops when full-system emulation is slow.

Setup outline:

1. Use Docker Buildx with linux/arm64 target.
2. Build and run your binary in an ARM64 container.
3. Use this for quick smoke tests; use Option A for full benchmark runs.

Output:

- Fast architecture validation path between major emulator runs.

Decision guidance

- If your immediate goal is deployment realism, choose Option A first.
- If your immediate goal is code iteration speed, keep Option B in parallel.

- Best practice: Use both.
-

7) How to Measure Resource Usage on Raspberry Pi (and emulator)

Measurement layers

1. In-application metrics (already present in your codebase)
 - Use your experiment monitor outputs for:
 - total runtime
 - CPU min/avg/peak
 - RAM min/avg/peak
 - per-row latency stats
2. OS-level metrics
 - Use external tools to validate process-level numbers:
 - `/usr/bin/time -v`
 - `pidstat -r -u -h -p <pid> 2`
 - `vmstat 2`
3. Device-health metrics (on real Pi)
 - Track thermal and throttling state:
 - `vcgencmd measure_temp`
 - `vcgencmd get_throttled`

Minimal repeatable protocol

1. Set a fixed run profile (same data slice, seed, build type).
2. Warm up once, then run N=3 measured runs.
3. Record median and min/max for:
 - total runtime
 - 100-timestamp processing time
 - CPU avg/peak
 - RAM avg/peak
4. Save all raw logs plus summarized CSV/JSON.

Recommended acceptance thresholds for first milestone

- Must complete without OOM at 2 GB memory profile.
- RAM peak under 350 MB.
- CPU average under 65%.
- 100 timestamps under 120 s.

Current standing (based on 2 trusted baseline runs):

Criterion	Target	Current baseline	Status
---	---	---	---
No OOM at 2 GB profile	Must pass	Average RAM peak is 4518.762 MB	Not met
RAM peak	< 350 MB	4518.762 MB	Not met
CPU average (CCPU)	< 65%	12.710%	Met
100 timestamps time	< 120 s	0.591 s (from average total runtime scaling)	Met

Note: this standing is from current x86 baseline runs, not yet from ARM/Raspberry Pi hardware or ARM emulation.

Stretch targets (competitive direction):

- RAM peak near or below 250 MB.
 - CPU average near or below 45%.
 - 100 timestamps near or below 80 s.
-

8) Risks and Mitigations

1. Emulator-performance mismatch vs real Pi
 - Mitigation: treat emulator timing as directional only; reserve hardware validation phase.
 2. Dependency friction for htm.core on ARM
 - Mitigation: lock compiler and dependency versions early; preserve working container image.
 3. Metric inconsistency
 - Mitigation: benchmark contract and fixed reporting template from week 1.
 4. Time pressure near deadline
 - Mitigation: keep May 28 to June 1 as hard buffer; cut low-value tuning if needed.
-

9) Immediate Next 7-Day Task List

1. Finalize benchmark contract and success thresholds.
 2. Capture fresh x86 baseline artifacts with current code.
 3. Stand up QEMU ARM64 environment and validate SSH workflow.
 4. Produce first ARM/emulated end-to-end run.
 5. Generate first HTM vs competitor comparison sheet (runtime, CPU, RAM, SWAT metrics).
 6. Open optimization backlog ranked by expected impact.
-

10) Definition of Done by June 1

1. End-to-end HTM SWAT run on ARM emulation, documented and reproducible.
2. Resource and timing report with 3-run stability summary.
3. Competitor comparison table including Raspberry Pi-relevant metrics.
4. Real-device deployment and measurement checklist ready for execution day.
5. Final technical report and reproducible command set committed.